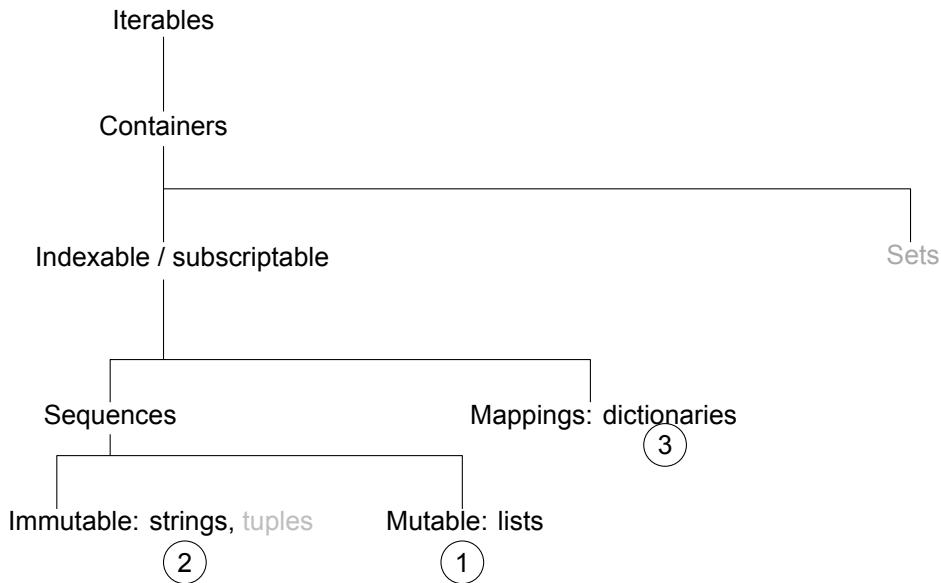


Python: container types (lists, strings, dictionaries)

Intro. to Computing

`https://mandarm.github.io/computing.html`

Container types



Container types

■ Lists: **mutable** sequence

- elements accessed using *indices*, and **can be changed**
- creating lists

```
l = [ 11, "y", 7.4 ]  
l = []           # empty list  
l = list(range(0,100,10)) # generally, list(iterable)  
numbers = [10, 16, -2, 22, -98]
```

LATER: using list comprehension

■ Strings (str) : **immutable** sequence

- elements accessed using *indices*, but **cannot be changed**

■ Dictionaries { 1:11, 2:"y", 3:7.4, 3.14:"pi" }

- elements accessed using *keys* (must be distinct)

■ Tuples (11, "y", 7.4)

■ Sets { 11, "y", 7.4 } — *not "subscriptable" (indexable)*

Lists: operations

(Applicable to immutable sequences also)

Operation	Result
<code>x in s</code>	True if an item of <code>s</code> is equal to <code>x</code> , else False ^{***}
<code>x not in s</code>	False if an item of <code>s</code> is equal to <code>x</code> , else True ^{***}
<code>s + t</code>	concatenation of <code>s</code> and <code>t</code> , does not change <code>s</code> , <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to concatenating <code>s</code> <code>n</code> times ^{††}
<code>len(s)</code>	length of <code>s</code>

^{***} For strings, `x` can also be a substring, e.g.,
`"gg" in "eggs"` `"ana" in "banana"`

^{††} **LATER:** may lead to unexpected behaviour

Exercise: Try using a negative value for `n`.

Lists: indexing and slicing

Indexing

negative indexes

s =

-n	-(n+1)	...	-2	-1
[10,	20,	...	40,	50,]
0	1	...	n-2	n-1

positive indexes

NOTE:
-0 ≡ 0

Slicing: `s[start:end:step]`

- start: starting index, **default** 0, *included* in slice
- end: ending index, **default** `len(s)`, *excluded* from slice
- step: skip step elements (step must be non-zero)
- Replace out of range values by 0 or `len(s)`, as appropriate.

Correction

List slicing with negative start, stop, step

- If start (or end) is negative, replace it by `len(l) + start` (or `len(l) + end`).
- If step is positive, and `start >= end`, slice is empty.
- If step is negative,
 - start must be greater than end; (`start <= end` $\Rightarrow$ slice is empty)
 - elements are listed in reverse order.
Example: `[22, 16, 10, -2, -98][10:0:-1]`
 $\rightarrow$ `[-98, -2, 10, 16]`

Question: how to print the entire list in reverse order?

If start or end are omitted or `None`, they become *end values* (which end depends on the sign of step).

Lists: more operations

(Applicable to mutable sequences only)

Operation	Result
<code>s[i] = x</code>	item <code>i</code> of <code>s</code> is replaced by <code>x</code>
<code>del s[i]</code>	removes item <code>i</code> of <code>s</code>
<code>s[i:j] = t</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of the <code>t</code>
<code>del s[i:j:k]</code>	removes the elements of <code>s[i:j:k]</code> from the list, <code>:k</code> optional same as <code>s[i:j:k] = []</code>
<code>s[i:j:k] = t</code>	elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code> <code>:k</code> optional, but if not 1, <code>s[i:j:k]</code> and <code>t</code> must have same length
<code>s += t</code>	extends <code>s</code> with contents of <code>t</code>
<code>s *= n</code>	updates <code>s</code> with its contents repeated <code>n</code> times

Lists: some useful functions I

(Applicable to immutable sequences also)

- `s.count(val)`: number of occurrences of `val` in sequence `s`

- `s.index(val)`

 - `s.index(value, start)`

 - `s.index(value, start, stop)`

index of the **first** occurrence of `value` in sequence `s`, but **ERROR** if it does not occur

LATER: exceptions (errors) and how to handle them

Usage:

```
s = [ 0, 1, 2, 3, 2, 0, 1, 0, 3, 2]
s.count(0)
[ 0, 1, 2, 3, 2, 0, 1, 0, 3, 2].count(3)
```

Lists: some useful functions II

(Applicable to mutable sequences only)

- `s.append(value)`
- `s.pop(index)`: index optional, **default** is -1
get the item at index and remove it from the sequence.
- `s.remove(value)`: remove the first item that matches value, but **ERROR** if it does not occur
- `s.extend(sequence)`
- `s.insert(index, value)`
- `s.reverse()`, `reversed(s)`: **NOTE the difference!**

Lists: more useful functions

If the elements can be compared

- `min(l)` smallest item of `l`
- `max(s)` largest item of `s`
- `sort()`, `sorted()`
- `sum()` for numeric types

```
numbers = [10, 16, -2, 22, -98]
sorted(numbers)      # [-98, -2, 10, 16, 22]
reversed(numbers)    # <list_reverseiterator object at ...>
numbers.sort()       # changes IN PLACE
numbers.reverse()    # changes IN PLACE
```

Note the difference between `sorted()` (returns a list) and `reversed()` (returns an iterable).

Strings (`str` type)

Creating strings

- Single, double, triple quotes
- `str()` function applied to any variable or expression
(creates printable string representation of the variable / expression)
- Two adjacent string literals separated by only whitespace automatically joined into single string literal

```
s1 = 'allows embedded "double" quotes'
s2 = "allows embedded 'single' quotes"
s3 = '''Three single
(allows multiple embedded newlines and ") quotes'''
s4 = """Three double
(allows multiple embedded newlines and ') quotes"""
s5 = str([1, 2, 3, 4])
s6 = str({"sky" : "blue", "grass" : "green", "blood" : "red"})
s7 = "hello" ' world'
```

Digression: escape sequences, 'raw' strings

```
>>> s4
"Three double \n(allows multiple embedded newlines and ') quotes"
>>> print(s4)
>>> s = 'hi\n'
>>> print(s)
>>> len(s)
```

Escape sequences: character sequences starting with a backslash (\) that denote some other character that may be hard to type / represent directly

Examples:

- `\t` $\equiv$ tab (somewhat useful for lining things up in columns)
- `\r` $\equiv$ carriage return (takes cursor back to beginning of line)
(try `print("hello\nworld")` and `print("hellooooo\rworld")`)

Raw strings

- strings prefixed with `r`, e.g., `r"hi\n"`
- escape sequences treated literally, do **NOT** have special meaning
(try `print(r"hello\nworld")` and `print(r"hellooooo\rworld")`)

Lists: operations

(Applicable to immutable sequences also)

Operation	Result
<code>x in s</code>	True if an item of <code>s</code> is equal to <code>x</code> , else False ^{***}
<code>x not in s</code>	False if an item of <code>s</code> is equal to <code>x</code> , else True ^{***}
<code>s + t</code>	concatenation of <code>s</code> and <code>t</code> , does not change <code>s</code> , <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to concatenating <code>s</code> <code>n</code> times ^{††}
<code>len(s)</code>	length of <code>s</code>

^{***} For strings, `x` can also be a substring, e.g.,
`"gg" in "eggs" "ana" in "banana"`

^{††} **LATER:** may lead to unexpected behaviour

Exercise: Try using a negative value for `n`.

String functions: character types

Digression: non-English languages and Unicode

- `s.isalpha()`: all characters in `s` are letters,
Examples: `'Letters and spaces'.isalpha()`,
`'LettersOnly'.isalpha()`
- `s.islower()`: all characters in `s` are lowercase
- `s.isupper()`: as above
- `s.isdigit()`, `s.isnumeric()`
- `s.isalnum()`
- `s.isspace()`: blanks, tabs, newlines, etc.

`s` must have at least one character of the desired type

String functions: changing case

Strings are **immutable**; all functions return a modified copy.

- `s.upper()`: all *cased* characters are converted to uppercase
Question: Is `s.upper().isupper()` guaranteed to return `True`?
- `s.lower()`: as above
- `s.swapcase()`: uppercase characters are converted to lowercase and vice versa
- `s.capitalize()`: first character is capitalised, rest is lowercased

String functions: searching

end, or both start and end may be omitted

- `in` operator, e.g., `"ana" in "banana"`
- `s.find(substring, start, end)`: returns lowest index in the string where substring `sub` is found, -1 if `sub` is not found

Question: What is the difference between `find` and `index`?

- `s.rfind(substring, start, end)`: as above, but returns highest index
- `s.startswith(prefix, start, end)`
 - possible to check whether `s` with any of a number of prefixes by using a *tuple* of strings in place of `prefix`
- `s.endswith(suffix, start, end)`: as above
- `s.count(sub, start, end)`: returns number of *non-overlapping* occurrences of `sub` in `s` (or appropriate slice of `s`)

Question: What happens when `sub` is the empty string (`""`)?

String functions: replacing

- `s.removeprefix(somestring)`: if the string starts with `somestring`, remove it from the returned copy, otherwise return the original string `s`
 - `s.removesuffix(somestring)`: as above
 - `s.strip()` or `s.strip("some string")`: any leading / trailing characters that are in the specified string is removed
(if no arguments are provided, leading and trailing *whitespace* is removed)
Examples: " abcd ".strip(), " abcd ".strip("dcba"),
"ad abcd bc".strip("dcba")
 - `s.lstrip()`, `s.rstrip()`: as above, but only leading (lstrip) OR trailing (rstrip) characters are removed
- Question:** What is the difference between `<lstrip(), removeprefix()>` and `<rstrip(), removesuffix()>`?
- `s.replace(oldstring, newstring, count)`: occurrences of `oldstring` are replaced by `newstring`
 - if `count` (optional) is not specified or negative, all occurrences are replaced; otherwise only the first `count` occurrences are replaced

Question: What happens if `oldstring` is ""?

String functions: splitting I

- `s.partition(separator-string)`: split the string at the *first* occurrence of separator-string; return a tuple containing 3 strings: (a) part before the separator, (b) separator, (c) part after the separator.
 - separator not found $\Rightarrow$ tuple contains `s`, `""` and `""`
- `s.rpartition(separator-string)`: as above, but use the *last* occurrence of separator-string

Digression: tuples

- Like lists, but *immutable*
- Use round brackets () instead of square brackets []

```
>>> t1 = ('a', 'b')
>>> t2 = tuple([1, 2, 3])
>>> t3 = (t1[1], t2[0])
```

String functions: splitting II

- `s.split(seperator-string)` can specify maximum number of pieces
Return list of “words” obtained by splitting `s` wherever `seperator-string` (must be non-empty) appears

Examples:

```
'1,,2,3'.split(',')  
# ['1', '', '2', '3']    (empty string because of consecutive commas)  
  
'1,2,,3,.'.split(',')    # ['1', '2', '', '3', '']  
'1<>2<>3<4'.split('<>') # ['1', '2', '3<4']  
  
# maxsplit=1 means at most 2 pieces  
'1,2,3'.split(',', maxsplit=1) # ['1', '2,3']
```

Question: What happens is separator-string does not occur in `s`?

Question: What happens if `s` is the empty string?

String functions: splitting III

- `s.split()` or `s.split(None)`
 - return list of “words” obtained by splitting `s` wherever *one or more whitespace characters* appear(s)
 - *leading or trailing whitespace is eaten up*

Examples:

```
'1 2 3'.split()           # ['1', '2', '3']  
'  1  2  3  '.split()    # ['1', '2', '3']  
'1 2 3'.split(maxsplit=1) # ['1', '2 3']
```

Homework: find out about `s.splitlines()`

String functions: joining

`s.join(iterable)`: return string formed by concatenating elements of iterable (must be strings) in iterable, with `s` inserted between successive elements

```
>>> ' followed by '.join(['abc', 'def', 'ghi'])  
>>> '-'.join('Python')      # 'P-y-t-h-o-n'
```

Homework: String functions related to formatting

- `s.zfill(width)`
- `s.ljust(width, fillchar)`,
`s.rjust(width, fillchar)`,
`s.center(width, fillchar)`
- `s.format()`

Dictionaries (`dict` type)

Dictionaries / mappings

- Set of *key-value* pairs
- Keys must be hashable
 - numbers, strings allowed as keys
 - lists, dictionaries not allowed as keys
- Creating dictionaries

```
d1 = { 2026: "1st year", 2025: "2nd year", 2024: "3rd year"}  
d2 = { "1st year": 2026, "2nd year": 2025, "3rd year": 2024}  
d3 = {'one': 1, 'two': 2, 'three': 3}
```

each item must have exactly 2 elements

```
d4 = dict([(1, 'jan'), (2, 'feb'), ])
```

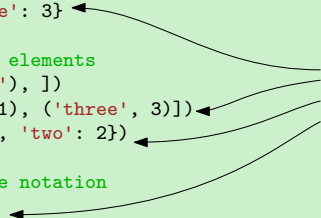
```
d5 = dict([('two', 2), ('one', 1), ('three', 3)])
```

```
d6 = dict({'three': 3, 'one': 1, 'two': 2})
```

also possible to use key=value notation

```
a = dict(one=1, two=2, three=3)
```

All these are
equal



hashable

An object is hashable if it has a hash value which never changes during its lifetime (it needs a `__hash__()` method), and can be compared to other objects (it needs an `__eq__()` method). Hashable objects which compare equal must have the same hash value.

Hashability makes an object usable as a dictionary key and a set member, because these data structures use the hash value internally.

Most of Python's immutable built-in objects are hashable; mutable containers (such as lists or dictionaries) are not; immutable containers (such as tuples and frozensets) are only hashable if their contents are hashable. Objects which are instances of user-defined classes are not hashable by default. They all compare unequal (except with themselves) and their hash value is derived from their `id()`.

Accessing values

- Use *key* as index to access *value*

```
print(d1[2026], d1[2024], d2["2nd year"])  
print(d3[1])
```

- Values that compare equal (such as 1, 1.0, and True) can be used interchangeably to index the same dictionary entry.
- Key must exist in the dictionary (as a key)
- Keys have to be distinct

Exercise: Try the following.

```
>>> d3[0]  
>>> d3[1.0]  
>>> d3[True]  
>>> d1 = { 2026: "1st year", 2026: "2nd year", 2024: "3rd year"}  
>>> d3 = dict([(1, 'jan'), (2, 'feb'), (1, 'January')])
```

Basic operations/functions

- `d.get(key, default-value)`: like `d[key]` but never returns error
 - `default-value` is optional
 - if key is not in `d`, return `default-value`, or `None` if default is not provided
- `d.setdefault(key, default-value)`
 - if key is in `d`, return its value;
 - otherwise, insert key with `default-value` and return `default-value`
 - if `default-value` (optional) is not provided, use `None`
- `d[key] = value` (key does not have to be an existing key in `d`)
- `del d[key]`: remove entry corresponding to key (*must exist!*) from `d`
- `d.clear()`: remove *all* entries in `d`
- `key in d`, `key not in d`, `not key in d`
- `list(d)`: return list of all *keys* currently in `d`
- `len(d)`: number of items in the dictionary `d`